C# اليوم 7 · الأسبوع 2 · ساعتان · دورة أساسيات البرمجة

الدوال والقوائم List ومدخل للكائنات

أهداف التعلم

في نهاية الدرس يستطيع المتدرب أن:

- يقسّم البرنامج إلى دوال (Methods) تستقبل مدخلات وتُرجع نتيجة.
- يستخدم List لإضافة العناصر وحذفها بمرونة.
- يعرّف كلاسًا بسيطًا Student بخصائص وينشئ منه كائنات.
- يتعامل مع الإدخال الخاطئ باستخدام int.TryParse.

توزيع وقت الحصة

0:00 – 0:10	مراجعة سريعة لدرس الأمس وحل الواجب
0:10 – 0:50	شرح الدرس مع أمثلة حيّة يكتبها المدرب والمتدربون معًا
0:50 – 1:00	استراحة
1:00 – 1:40	تمارين عملية فردية (المدرب يتنقل بين المتدربين الستة)
1:40 – 1:55	تصحيح جماعي ومناقشة الأخطاء الشائعة
1:55 – 2:00	ملخص الدرس + اختبار ذاتي سريع + الواجب

شرح الدرس

1. الدوال (Methods)

الدالة مجموعة أوامر لها اسم، نكتبها مرة ونستدعيها كثيرًا. `void` تعني أنها لا تُرجع قيمة.

```C#
static void Greet(string name)
{
    Console.WriteLine($"يا أهلًا {name}");
}

static bool IsPassed(int score)
{
    return score ⩾ 60;
}

Greet("مريم");
Console.WriteLine(IsPassed(75));   // True
Console.WriteLine(IsPassed(40));   // False
```

ماذا يفعل هذا الكود؟ برنامج يعرّف دالتين: الأولى تطبع ترحيبًا بالاسم، والثانية تفحص هل الدرجة ناجحة. يرى الطالب «أهلًا يا مريم» ثم True ثم False.

`static void Greet(string name)`	هنا نعرّف دالة (method) اسمها Greet، والدالة مجموعة أوامر لها اسم نستدعيها متى شئنا. void تعني أنها لا تُرجع أي نتيجة، وما بين القوسين (string name) هو مُدخَل (parameter) نصي تستقبله. وstatic تعني أنها دالة مستقلة لا تستخدم متغيرات من خارجها.
`{`	الأقواس المعقوصة { و } تجمع أوامر الدالة في كتلة واحدة، فكل ما بينها هو جسم الدالة.
`Console.WriteLine($"أهلًا يا {name}");`	هذا هو عمل الدالة: تطبع ترحيبًا يحتوي على الاسم الذي أُرسل إليها في name.
`static bool IsPassed(int score)`	دالة ثانية اسمها IsPassed تستقبل عددًا صحيحًا score. النوع bool قبل اسمها يعني أنها تُرجع نتيجة منطقية: true (صحيح) أو false (خطأ).
`return score ⩾ 60;`	الكلمة return تُنهي الدالة وتُعيد النتيجة لمن استدعاها. هنا نُرجع ناتج المقارنة score >= 60 مباشرة، فيكون true إذا كانت الدرجة 60 أو أكثر وإلا false.
`Greet("مريم");`	نستدعي (نشغّل) الدالة Greet ونرسل لها النص "مريم"، فيصبح name داخلها يساوي مريم ويُطبع «أهلًا يا مريم».
`Console.WriteLine(IsPassed(75));` `// True`	نستدعي IsPassed بالدرجة 75، فتُرجع true لأن 75 >= 60، ثم نطبع النتيجة فتظهر True.
`Console.WriteLine(IsPassed(40));` `// False`	هذه المرة الدرجة 40 أقل من 60، فتُرجع الدالة false ويظهر False. هكذا نعيد استخدام الدالة نفسها بقيم مختلفة.

2. القوائم List

المصفوفة حجمها ثابت، أما `List` فتكبر وتصغر. أهم العمليات: `Add` و`Remove` و`Count` .

C#

```
List<string> names = new List<string>();
names.Add("سارة");
names.Add("خالد");
names.Add("مريم");
names.Remove("خالد");

Console.WriteLine($"عدد الطلاب: {names.Count}");
foreach (string n in names)
    Console.WriteLine("- " + n);
```

```
Output ▸ 2 :عدد الطلاب
- سارة
- مريم
```

ماذا يفعل هذا الكود؟ برنامج ينشئ قائمة أسماء، يضيف إليها ثلاثة أسماء ثم يحذف واحدًا، ويطبع عدد الطلاب (2) والأسماء المتبقية.

`List<string> names = new List<string>();`	List<string> قائمة من النصوص، وتختلف عن المصفوفة بأن حجمها يكبر ويصغر عند الإضافة والحذف. ما بين < > هو نوع العناصر، والكلمة new تنشئ قائمة جديدة فارغة.
`names.Add("سارة");`	الدالة Add تضيف عنصرًا إلى آخر القائمة؛ وهذا السطر والسطران التاليان يضيفون سارة ثم خالد ثم مريم بالترتيب.
`names.Remove("خالد");`	الدالة Remove تبحث عن العنصر "خالد" وتحذفه من القائمة، فتتقدم العناصر التي بعده مكانه.
`Console.WriteLine($"عدد الطلاب: {names.Count}");`	الخاصية Count تعطي عدد العناصر الموجودة الآن في القائمة، وهي 2 بعد الحذف. (في المصفوفة نستخدم Length، وفي List نستخدم Count).
`foreach (string n in names)`	foreach تمر على كل اسم في القائمة بالترتيب، ويحمل المتغير n الاسم الحالي. جسم الحلقة هو السطر التالي فقط لذلك لا نحتاج { }.
`Console.WriteLine("- " + n);`	المعامل + مع النصوص يلصقها معًا، فيُطبع كل اسم مسبوقًا بشرطة مثل «- سارة».

3. الكلاس والكائن

الكلاس **قالب** يصف شيئًا (مثل الطالب)، والكائن **نسخة** فعلية منه. هذا هو أساس البرمجة الكائنية، وسنستخدمه لتمثيل صفوف قاعدة البيانات.

C#

```
var list = new List<Student>
{
    new Student { Name = "سارة", Grade = 95 },
    new Student { Name = "خالد", Grade = 55 }
};

foreach (var s in list)
    Console.WriteLine($"{s.Name}: {s.Grade} - {(s.Grade ⩾ 60 ? "راسب" : "ناجح")}");

class Student
{
    public string Name { get; set; }
    public int Grade { get; set; }
```

```
}
```

ماذا يفعل هذا الكود؟ برنامج يعرّف صنفًا Student له اسم ودرجة، وينشئ قائمة بطالبين، ثم يطبع لكل طالب اسمه ودرجته وهل هو ناجح أم راسب.

`var list = new List<Student>`	ننشئ قائمة عناصرها من النوع Student (صنف سنعرّفه في آخر الملف). الكلمة var تجعل #C يستنتج نوع المتغير تلقائيًا من الجهة اليمنى، فلا نحتاج كتابة List<Student> مرتين.
`{`	الأقواس المعقوصة هنا تجمع العناصر الأولى التي نضعها في القائمة لحظة إنشائها، مفصولة بفواصل.
`new Student { Name = "سارة", Grade = 95 },`	new Student ينشئ كائنًا (object) جديدًا من الصنف Student، أي طالبًا فعليًا، وما بين { } يعطي خصائصه قيمها الأولى: الاسم سارة والدرجة 95.
`new Student { Name = "خالد", Grade = 55 }`	ننشئ طالبًا ثانيًا اسمه خالد ودرجته 55. لا توجد فاصلة بعده لأنه آخر عنصر في القائمة.
`foreach (var s in list)`	نمر على كل طالب في القائمة، ويحمل المتغير s الطالب الحالي؛ وvar هنا تعني أن نوعه Student.
`Console.WriteLine($"{s.Name}: {s.Grade} - {(s.Grade >= 60 ? "ناجح" : "راسب")}");`	النقطة في s.Name تعني «خاصية Name الخاصة بهذا الطالب». أما الشرط المختصر ? : فيعمل هكذا: إذا كان s.Grade >= 60 صحيحًا يعطي "ناجح" وإلا يعطي "راسب"، ونضعه بين أقواس ( ) داخل { } لأنه يحتوي على النقطتين :.
`class Student`	الكلمة class تعرّف صنفًا، وهو قالب أو مخطط يصف شكل نوع جديد من البيانات؛ هنا كل طالب له اسم ودرجة. في #C يجب أن تأتي تعريفات الأصناف بعد أوامر البرنامج الرئيسية في الملف.
`public string Name { get; set; }`	هذه خاصية (property) اسمها Name من النوع string. الكلمة public تعني أنه يمكن الوصول إليها من خارج الصنف، و{ get; set; } تعني أنه يمكن قراءة قيمتها (get) وتغييرها (set).
`public int Grade { get; set; }`	خاصية ثانية اسمها Grade من النوع int لتخزين درجة الطالب، ويمكن قراءتها وتغييرها أيضًا.

في مشاريع top-level statements يكون تعريف الكلاس في نهاية الملف بعد الأوامر.

4. حماية البرنامج من الإدخال الخاطئ

إذا كتب المستخدم حروفًا بدل رقم، فإن `int.Parse` يوقف البرنامج بخطأ. `int.TryParse` يحاول التحويل ويُرجع true أو false.

```
C#
```

```
Console.Write("العمر: ");
if (int.TryParse(Console.ReadLine(), out int age))
    Console.WriteLine($"عمرك {age}");
else
    Console.WriteLine("الرجاء إدخال رقم صحيح");
```

ماذا يفعل هذا الكود؟ برنامج يطلب العمر ويتأكد أن المدخل رقم صحيح بأمان: إن كان رقمًا يطبع «عمرك ...»، وإلا يطبع رسالة تطلب إدخال رقم صحيح بدل أن يتوقف بخطأ.

`Console.Write("العمر: ");`	نطبع سؤال العمر دون الانتقال إلى سطر جديد.
`if (int.TryParse(Console.ReadLine(), out int age))`	int.TryParse «تحاول» تحويل النص إلى عدد صحيح: تُرجع true إذا نجحت وfalse إذا فشلت، دون أن يتوقف البرنامج كما يحدث مع int.Parse. والكلمة out int age تعني أن الدالة ستضع الرقم الناتج في متغير جديد اسمه age ننشئه في المكان نفسه.
`Console.WriteLine($"عمرك {age}");`	هذا السطر يُنفَّذ فقط إذا نجح التحويل، فنطبع العمر المخزّن في age.
`else`	else تعني «وإلا»: الأمر الذي بعدها يُنفَّذ عندما يكون شرط if خاطئًا، أي عندما يكتب المستخدم شيئًا ليس رقمًا صحيحًا.
`Console.WriteLine("الرجاء إدخال رقم صحيح");`	نطبع رسالة لطيفة تخبر المستخدم بالخطأ بدلًا من أن ينهار البرنامج.

التمارين العملية

تمرين 1: دالة المساحة

سهل

اكتب دالة CircleArea تستقبل نصف القطر وتُرجع مساحة الدائرة (Math.PI × r × r)، واستدعها لثلاث قيم.

الحل المقترح:

C#

```
static double CircleArea(double r)
{
    return Math.PI * r * r;
}

Console.WriteLine(CircleArea(1).ToString("F2"));
Console.WriteLine(CircleArea(2.5).ToString("F2"));
Console.WriteLine(CircleArea(10).ToString("F2"));
```

ماذا يفعل هذا الكود؟ برنامج يعرّف دالة تحسب مساحة الدائرة من نصف قطرها، ثم يطبع مساحة ثلاث دوائر مقرّبة إلى

رقمين عشريين: 3.14 و19.63 و314.16.

`static double CircleArea(double r)`	نعرّف دالة اسمها CircleArea تستقبل نصف القطر r كعدد عشري (double)، وتُرجع نتيجة من النوع double أيضًا. الكلمة static تعني أنها دالة مستقلة لا تعتمد على متغيرات خارجها.
`{`	الأقواس المعقوصة { و } تجمع أوامر الدالة في كتلة واحدة هي جسم الدالة.
`return Math.PI * r * r;`	Math.PI ثابت جاهز في C# قيمته π تقريبًا 3.14159، ومساحة الدائرة = π × نصف القطر × نصف القطر. الكلمة return تُرجع هذه النتيجة لمن استدعى الدالة.
`Console.WriteLine(CircleArea(1).ToString("F2"));`	نستدعي الدالة بنصف قطر 1، ثم الدالة ToString("F2") تحوّل الرقم الناتج إلى نص برقمين بعد الفاصلة العشرية، فيظهر 3.14.
`Console.WriteLine(CircleArea(2.5).ToString("F2"));`	نعيد استخدام الدالة نفسها بنصف قطر 2.5، فيظهر 19.63. هذه فائدة الدوال: نكتب الحساب مرة ونستخدمه مرات.
`Console.WriteLine(CircleArea(10).ToString("F2"));`	ومرة ثالثة بنصف قطر 10، فيظهر 314.16.

تمرين 2: قائمة طلاب تفاعلية

متوسط

برنامج بقائمة: 1 إضافة اسم، 2 عرض الأسماء، 3 حذف اسم، 0 خروج. استخدم while وswitch وList.

الحل المقترح:

C#

```
var names = new List<string>();
string choice = "";

while (choice ≠ "0")
{
    Console.WriteLine("\n1 خروج 0 | حذف 3 | عرض 2 | إضافة");
    choice = Console.ReadLine();
    switch (choice)
    {
        case "1":
            Console.Write("الاسم: ");
            names.Add(Console.ReadLine());
            break;
        case "2":
            for (int i = 0; i < names.Count; i++)
                Console.WriteLine($"{i + 1}. {names[i]}");
            break;
        case "3":
```

```
            Console.Write("الاسم المراد حذفه: ");
            if (names.Remove(Console.ReadLine())) Console.WriteLine("تم الحذف");
            else Console.WriteLine("غير موجود");
            break;
    }
}
```

ماذا يفعل هذا الكود؟ برنامج قائمة أسماء تفاعلي يعرض قائمة خيارات متكررة: إضافة اسم، عرض الأسماء مرقّمة، حذف اسم، أو الخروج بكتابة 0.

`var names = new List<string>();`	ننشئ قائمة نصوص فارغة باسم names لتخزين الأسماء، وvar تجعل C# يستنتج أن نوعها List<string>.
`string choice = "";`	متغير نصي يحفظ اختيار المستخدم من القائمة، ونبدؤه بنص فارغ حتى تدخل الحلقة أول مرة.
`while (choice ≠ "0")`	الحلقة while تستمر في عرض القائمة ما دام المستخدم لم يختر "0" (الخروج).
`{`	الأقواس المعقوصة { و } تجمع الأسطر التي تتكرر داخل الحلقة في كتلة واحدة.
`Console.WriteLine("\n1 2 | إضافة 3 | حذف 0 | خروج");`	نطبع قائمة الخيارات. الرمز \n داخل النص يعني «سطر جديد»، فيترك سطرًا فارغًا قبل القائمة لتكون أوضح.
`choice = Console.ReadLine();`	نقرأ اختيار المستخدم كنص ونخزّنه في choice.
`switch (choice)`	switch يقارن قيمة choice بعدة احتمالات، وكل احتمال يبدأ بالكلمة case؛ وهو بديل أوضح من كتابة if و else if كثيرة.
`case "1":`	هذا الجزء يُنفَّذ إذا كانت قيمة choice تساوي "1" أي اختيار الإضافة.
`Console.Write("الاسم: ");`	نطلب من المستخدم كتابة الاسم المراد إضافته.
`names.Add(Console.ReadLine());`	نقرأ الاسم ونضيفه مباشرة إلى آخر القائمة باستخدام Add.
`break;`	break تُنهي هذا الcase وتخرج من switch. في C# يجب أن ينتهي كل case بأمر مثل break حتى لا يكمل التنفيذ إلى الحالة التالية.
`case "2":`	هذا الجزء للاختيار "2" أي عرض الأسماء.
`for (int i = 0; i < names.Count; i++)`	حلقة for تمر على مواقع القائمة من 0 حتى names.Count - 1، وCount هو عدد الأسماء الحالي.
`Console.WriteLine($"{i + 1}. {names[i]}");`	نطبع رقم الاسم i + 1 (لنبدأ العدّ من 1 للمستخدم) ثم الاسم نفسه names[i]، فالقائمة مثل المصفوفة نصل إلى عناصرها بالفهرس بين [ ].

`case "3":`	هذا الجزء للاختيار "3" أي حذف اسم.
`Console.Write("الاسم المراد حذفه: ");`	نطلب من المستخدم كتابة الاسم الذي يريد حذفه.
`if (names.Remove(Console.ReadLine())) Console.WriteLine("تم الحذف");`	الدالة Remove تحاول حذف الاسم وتُرجع true إذا وجدته وحذفته، وfalse إذا لم يكن موجودًا؛ لذلك نضعها مباشرة داخل if ونطبع «تم الحذف» عند النجاح.
`else Console.WriteLine("غير موجود");`	وإذا أرجعت Remove القيمة false، أي أن الاسم ليس في القائمة، نطبع «غير موجود».

تمرين 3: تحدٍّ: كلاس المنتج

تحدٍّ

عرّف كلاس Product (Name, Price, Qty) ودالة Total تُرجع Price × Qty. أنشئ 3 منتجات في List واطبع إجمالي قيمة المخزون.

الحل المقترح:

C#

```
var items = new List<Product>
{
    new Product { Name = "قلم", Price = 0.5, Qty = 100 },
    new Product { Name = "دفتر", Price = 1.2, Qty = 40 },
    new Product { Name = "حقيبة", Price = 8, Qty = 5 }
};

double all = 0;
foreach (var p in items)
{
    Console.WriteLine($"{p.Name}: {p.Total()}");
    all += p.Total();
}
Console.WriteLine($"الإجمالي: {all}");

class Product
{
    public string Name { get; set; }
    public double Price { get; set; }
    public int Qty { get; set; }
    public double Total() => Price * Qty;
}
```

ماذا يفعل هذا الكود؟ برنامج مخزن صغير: يعرّف صنف منتج له اسم وسعر وكمية ودالة تحسب قيمته الكلية، ثم يطبع قيمة كل منتج والإجمالي لكل المنتجات.

`var items = new List<Product>`	ننشئ قائمة عناصرها من النوع Product (صنف معرَّف في آخر الملف)، وvar تجعل #C يستنتج نوع المتغير تلقائيًا.
`{`	الأقواس المعقوصة هنا تجمع المنتجات الأولى التي نضعها في القائمة لحظة إنشائها، مفصولة بفواصل.
`new Product { Name = "قلم", Price = 0.5, Qty = 100 },`	new Product ينشئ كائنًا جديدًا من الصنف Product، ونعطي خصائصه قيمها بين { }: الاسم قلم والسعر 0.5 والكمية 100. والسطران التاليان ينشئان الدفتر والحقيبة بالطريقة نفسها.
`double all = 0;`	متغير عشري all يبدأ بصفر لنجمع فيه قيمة كل المنتجات.
`foreach (var p in items)`	نمر على كل منتج في القائمة، ويحمل المتغير p المنتج الحالي.
`Console.WriteLine($"{p.Name}: {p.Total()}");`	نطبع اسم المنتج ثم نستدعي دالته Total() التي تحسب قيمته الكلية. الأقواس () بعد Total ضرورية لأنها دالة تُستدعى وليست خاصية.
`all += p.Total();`	نضيف قيمة هذا المنتج إلى الإجمالي باستخدام =+ أي all = all + p.Total().
`Console.WriteLine($"الإجمالي: {all}");`	بعد انتهاء الحلقة نطبع مجموع قيم كل المنتجات.
`class Product`	نعرّف الصنف Product، وهو قالب يصف كل منتج: ما يحمله من بيانات (خصائص) وما يستطيع فعله (دوال). تأتي الأصناف بعد أوامر البرنامج الرئيسية في الملف.
`public string Name { get; set; }`	خاصية نصية Name لاسم المنتج؛ public تعني أنها متاحة من خارج الصنف، و{ get; set; } تعني أنها قابلة للقراءة والتغيير.
`public double Price { get; set; }`	خاصية Price لسعر الوحدة، ونوعها double لأن السعر قد يحتوي على كسور مثل 0.5.
`public int Qty { get; set; }`	خاصية Qty للكمية، ونوعها int لأن الكمية عدد صحيح.
`public double Total() ⇒ Price * Qty;`	دالة داخل الصنف اسمها Total تُرجع double. السهم => طريقة مختصرة تعني «أرجِع هذه النتيجة»، بدل كتابة { return Price * Qty; }، فتحسب السعر مضروبًا في الكمية لهذا المنتج.

اختبر نفسك (3 أسئلة)

1. الكلمة void تعني أن الدالة:

لا تُرجع قيمة | تُرجع رقمًا | فارغة من الأوامر

2. لإضافة عنصر إلى List نستخدم:

Push | Add | Insert فقط

3. الكلاس بالنسبة للكائن هو:

نسخة منه | القالب الذي يُصنع منه | متغير عادي

الإجابات: 1) لا تُرجع قيمة · 2) Add · 3) القالب الذي يُصنع منه

الواجب: جهّز تثبيت SQL Server Express وSSMS على جهازك قبل يوم 8 (الرابط في قسم الخطة).